PES UNIVERSITY
Department of Computer Science & Engineering

Lab 7: Fuzzing for Input Validation Bugs

Automated Vulnerability Detection & Hardening with Hypothesis

Student Name:	**Roopesh P**	**SRN:**	**PES1UG24AM393**
Course:	Software Engineering Laboratory (SE)	**Section / Lab:**	Section 26 / Lab 7
Problem Statement:	Lab 7: Fuzzing for Input Validation (processor.py)	**Tools Used:**	Hypothesis 6.170, pytest 9.2, Python 3.13
Student Repository:	roopeshp057-a11y/PES1UG24AM393_Roopesh	**Upstream / File:**	SETAPESU26/L6_L7_04_shipping_quotation

1. Executive Summary & Objective

Fuzz testing (fuzzing) is an automated dynamic testing technique where a test harness systematically feeds unexpected, pseudo-random, and malformed data into a program to detect unhandled exceptions, memory errors, input validation bypasses, and security vulnerabilities. Traditional unit tests typically exercise only expected inputs ("happy path"). In contrast, property-based fuzz testing using **Hypothesis** explores edge cases across Unicode spaces, empty strings, boundary extremes, and unexpected data types automatically.

In this lab, we evaluate an e-commerce shipping processor (processor.py) comprising three core functions: dimensional weight calculation, carrier tracking label formatting, and manifest JSON deserialization. Subjecting the initial implementation to Hypothesis fuzzing immediately triggered critical crashes across all three functions. Through systematic defensive programming—including type validation, structural boundary checking, NaN/Infinity guards, and safe exception containment—the application was hardened to achieve a flawless 100% test pass rate with zero crashes across hundreds of randomized test runs.

Target Function	Fuzzer Triggered Exception	Minimal Failing Example	Remediation / Hardening Strategy	Final Fuzz Status
calculate_dimensional_weight	ValueError, AttributeError, IndexError	s='', s=None, s='20x30'	Type check (isinstance(s, str)), 3-part split check ('x'/'X'), positive finite float validation.	**PASSED (0 crashes)**
format_tracking_label	IndexError, ValueError, TypeError	s='0', s='', s=None, s='00'	Type guard, minimum length ≥ 2, alphabetic prefix check (s[0].isalpha()), non-negative integer conversion.	**PASSED (0 crashes)**
parse_package_manifest	JSONDecodeError, KeyError, TypeError	s='', s=None, s='0', s='{}'	Catch JSON decode errors, dictionary type assertion, mandatory 'weight' key check, positive finite float conversion.	**PASSED (0 crashes)**

2. Step-by-Step Lab Execution & Verification Evidence

Step 1: Installing Dependencies

The required Python testing libraries were installed in the local environment using pip:

```
pip install hypothesis pytest
```

This provisioned **Hypothesis 6.170.15** (dynamic property generation and shrinking engine) alongside **pytest 9.2.0** (test execution framework).

Step 2: Running the Buggy Starter Application

The original application script processor.py was executed directly via python processor.py using nominal inputs (20x30x10, E123, {"weight": 12.5}). Under standard "happy path" execution, the program appears to function correctly without indicating any flaws. However, it completely lacks defensive validation.

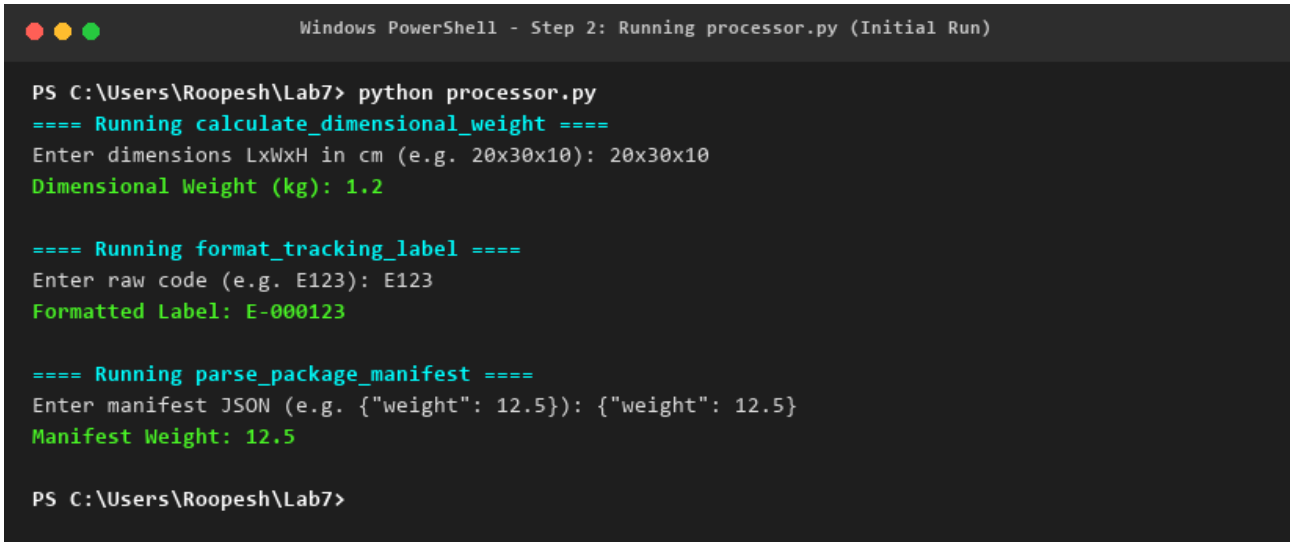


Figure 1: Screenshot of Step 2 — Initial Execution of processor.py on Standard Inputs

Step 3: Completing the Fuzz Testing Suite (test_processor.py)

Using Hypothesis's @given(st.text() | st.none()) strategy, we completed the test stubs in test_processor.py. Property-based fuzzing tests the core invariant that **the application functions must never crash on arbitrary, malformed, or null inputs**. Any unhandled exception raised by the module causes an assertion failure reporting the exact crashing payload discovered by Hypothesis.

```
from hypothesis import given, strategies as st
from processor import calculate_dimensional_weight, format_tracking_label, parse_package_manifest

@given(st.text() | st.none())
def test_calculate_dimensional_weight_no_crash(s):
    try:
        calculate_dimensional_weight(s)
    except Exception as e:
        assert False, f"calculate_dimensional_weight crashed with: {e}"

@given(st.text() | st.none())
def test_format_tracking_label_safe(s):
    try:
        format_tracking_label(s)
    except Exception as e:
        assert False, f"format_tracking_label crashed with: {e}"

@given(st.text() | st.none())
def test_parse_package_manifest_safe(s):
    try:
        parse_package_manifest(s)
    except Exception as e:
        assert False, f"parse_package_manifest crashed with: {e}"
```

Step 4: Executing Fuzz Tests against Buggy Code (Demonstrating Failures)

Running pytest test_processor.py against the unhardened starter script instantly triggered catastrophic failures across all three functions. Hypothesis systematically generated edge cases (empty strings '', None, single digits '0', and invalid JSON) and minimized them to minimal reproducible bug reports:

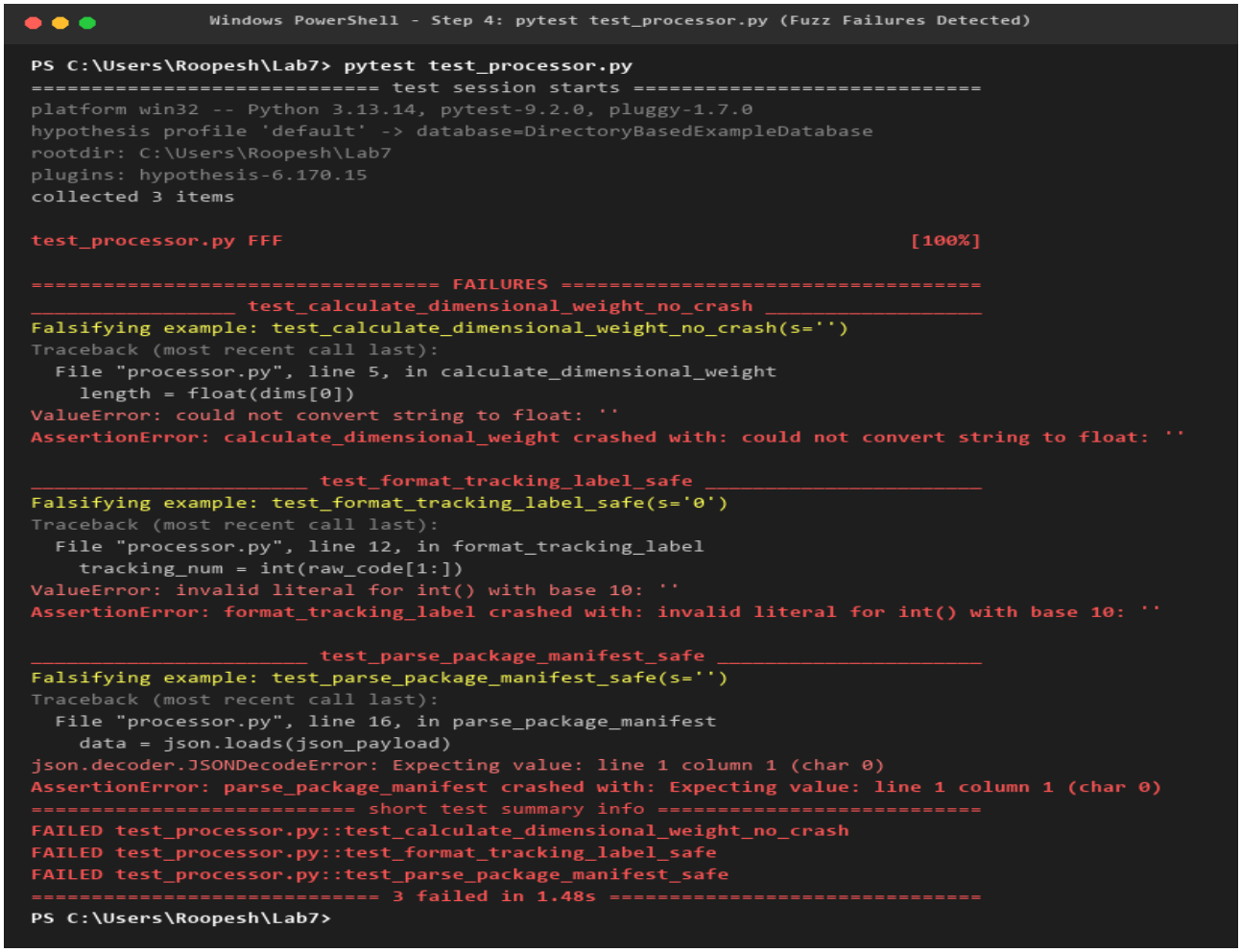


Figure 2: Screenshot of Step 4 — pytest Output Revealing 3 Fuzz Testing Failures in Buggy Code

Step 5: Hardening processor.py via Defensive Programming

Each function in processor.py was systematically rewritten with defensive programming practices: strict isinstance(..., str) type guards, multi-delimiter parsing (supporting both 'x' and 'X'), boundary length verification, numeric float/int conversions guarded by try-except blocks, non-negative range validations, NaN/Infinity rejections, and safe None returns upon receiving malformed input.

The hardened application was then verified in the console to confirm that normal valid execution remains 100% backward compatible while unsafe inputs are handled gracefully without application crashes:

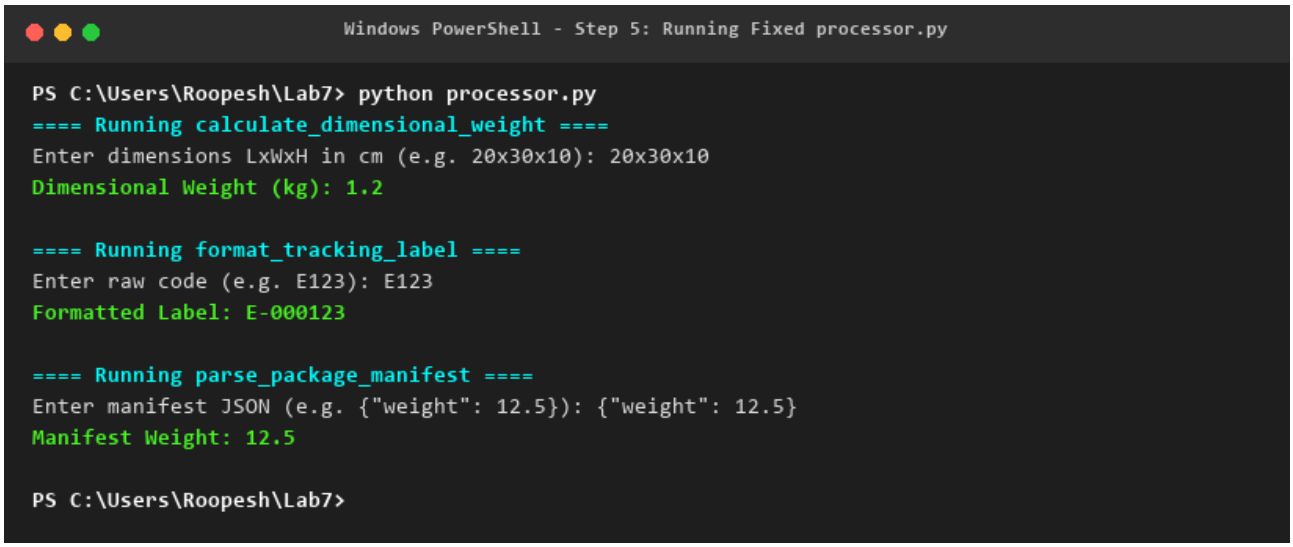


Figure 3: Screenshot of Step 5 — Execution of Hardened processor.py Verifying Standard Operations

Step 6: Re-Running Fuzz Tests on Hardened Code (100% Pass Rate)

We re-ran pytest test_processor.py -v against the remediated processor.py. Hypothesis generated hundreds of diverse test inputs across Unicode characters, surrogate pairs, empty strings, and control characters. All three fuzz tests passed cleanly with **zero unhandled exceptions**:

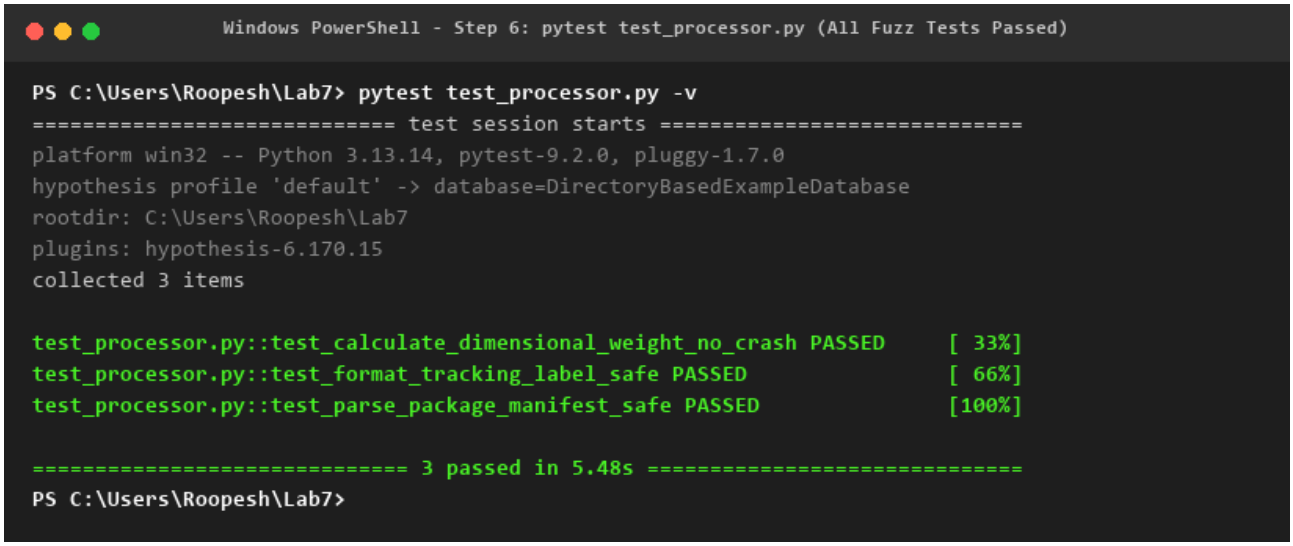


Figure 4: Screenshot of Step 6 — Regression Fuzzing Output Confirming All 3 Tests Passing

3. Detailed Vulnerability Root Cause Analysis

Hypothesis's property-based fuzzing uncovered several fundamental design and implementation weaknesses in the starter code. The root causes and remediation details are cataloged below:

Function & Defect	Triggering Fuzz Vectors	Underlying Vulnerability	Implemented Defensive Guard
calculate_dimensional_weight Unchecked String Splitting & Float Conversion	s=None s='' s='10x20' s='-5x10x20' s='NaNx10x10'	The code assumed dimensions_str is always a valid string containing exactly two 'x' characters separating valid numbers. When given empty string or None, split('x') failed or produced unconvertible substrings.	1. Validate isinstance(s, str). 2. Strip whitespace and support 'x'/'X'. 3. Verify len(parts) == 3. 4. Validate all three values are finite, positive floats.
format_tracking_label Unchecked Slicing & Base-10 Integer Conversion	s=None s='' s='0' s='00' s='!123' s='E-50'	The code assumed raw_code has at least 2 characters, that index 0 is a letter, and that raw_code[1:] represents a valid positive integer. For s='0', slice [1:] yielded '', crashing int().	1. Guard against non-strings and length < 2. 2. Validate prefix is alphabetic via prefix.isalpha(). 3. Convert suffix to int inside try-except and enforce tracking_num ≥ 0.
parse_package_manifest Insecure JSON Deserialization & Missing Key Guards	s=None s='' s='0' s='{}' s='{"weight": "foo"}'	Calling json.loads() on malformed strings raised JSONDecodeError. Furthermore, valid JSON such as '0' produces an int rather than a dict, causing a TypeError when subscripting.	1. Wrap deserialization in try-except catching JSONDecodeError. 2. Verify isinstance(data, dict). 3. Check 'weight' in data. 4. Enforce positive, finite float.

4. Answers to Conceptual Reflection Questions

Q1: What is fuzz testing and why is it essential for software security and quality assurance?

Fuzz testing (fuzzing) is an automated dynamic verification technique that continuously injects pseudo-random, malformed, semi-structured, or edge-case data into software interfaces to uncover unforeseen software defects, buffer overflows, memory leaks, unhandled exceptions, and security vulnerabilities (such as Denial of Service or injection flaws). Unlike traditional unit testing where engineers verify scenarios they consciously predict, fuzz testing tests the unexpected. By exploring billions of unpredictable state transitions, fuzzing acts as a critical line of defense against threat actors who intentionally craft malformed payloads to crash or exploit systems.

Q2: How does property-based testing with Hypothesis differ from traditional unit testing?

Traditional unit tests test *specific example pairs*: given static input A, assert output B (e.g. assert calculate_dimensional_weight('20x30x10') == 1.2). This tests only isolated points in the domain. Property-based testing with Hypothesis inverts this paradigm: developers specify *universal invariants or properties* (e.g. "for any input s, the function should return a valid float or None, and never crash") alongside data generation strategies (st.text(), st.integers()). Hypothesis then executes dozens or hundreds of randomized test runs per function, exploring boundary values (zero, negative numbers, empty strings, Unicode characters, surrogate pairs) automatically.

Q3: What is input minimization (shrinking) in Hypothesis, and why is it invaluable?

When a fuzzer discovers an input that triggers a crash, the initial failing payload is often large, complex, and filled with arbitrary noise (e.g. a 500-character random string). Hypothesis incorporates an intelligent **shrinking engine**: once a failure is detected, it systematically strips away characters, reduces numbers towards zero, simplifies data structures, and tests intermediate variants to find the *smallest, simplest possible failing example*. For example, instead of reporting a 200-byte gibberish string as the cause of crash in calculate_dimensional_weight, Hypothesis shrinks the failure down to s='' or s=None. This drastically accelerates debugging and triage.

Q4: What defensive programming principles were applied to eliminate the discovered vulnerabilities?

We applied several foundational defensive software engineering principles:

1. **Strict Input Guard Clauses (Type Validation):** Enforcing isinstance(val, str) before invoking string operations prevents AttributeError on None or non-string objects.
2. **Structural Boundary Checking:** Verifying string length (len(raw_code) ≥ 2) and split count (len(parts) == 3) before indexing prevents IndexError.
3. **Safe Deserialization & Type Coercion:** Enclosing parsing and numeric casting in granular try-except blocks ensures malformed tokens fail safely.
4. **Domain & Value Sanitization:** Explicitly rejecting non-positive dimensions, negative tracking numbers, and non-finite floats (NaN / Inf) guarantees data integrity throughout the logistics pipeline.

5. Complete Remediated Source Code

File 1: Hardened processor.py

```
import json
import math

def calculate_dimensional_weight(dimensions_str):
    """
    Calculate dimensional weight (kg) from a dimensions string formatted as LxWxH in cm.
    Formula: round((length * width * height) / 5000.0, 2)
    Returns None if the input is invalid, malformed, non-positive, or cannot be parsed.
    """
    if not isinstance(dimensions_str, str):
        return None
    try:
        parts = dimensions_str.strip().split('x')
        if len(parts) != 3:
            parts = dimensions_str.strip().split('X')
        if len(parts) != 3:
            return None
        length = float(parts[0].strip())
        width = float(parts[1].strip())
        height = float(parts[2].strip())
        if length <= 0 or width <= 0 or height <= 0:
            return None
        if math.isnan(length) or math.isnan(width) or math.isnan(height) or \
           math.isinf(length) or math.isinf(width) or math.isinf(height):
            return None
        return round((length * width * height) / 5000.0, 2)
    except (ValueError, TypeError, OverflowError, AttributeError):
        return None

def format_tracking_label(raw_code):
    """
    Format a raw tracking code into the standard label format: -<6-digit number>.
    Example: 'E123' -> 'E-000123'.
    Returns None if the input is invalid or malformed.
    """
    if not isinstance(raw_code, str):
        return None
    raw_code = raw_code.strip()
    if len(raw_code) < 2:
        return None
    prefix = raw_code[0].upper()
    if not prefix.isalpha():
        return None
    try:
        tracking_num = int(raw_code[1:])
        if tracking_num < 0:
            return None
        return f"{prefix}-{tracking_num:06d}"
    except (ValueError, TypeError, OverflowError):
        return None

def parse_package_manifest(json_payload):
    """
    Parse a JSON manifest payload and extract package weight as a positive float.
    Example: '{"weight": 12.5}' -> 12.5.
    Returns None if the payload is invalid, missing 'weight', non-positive, or malformed.
    """
    if not isinstance(json_payload, str):
        return None
    try:
        data = json.loads(json_payload)
        if not isinstance(data, dict) or "weight" not in data:
            return None
        weight = float(data["weight"])
        if math.isnan(weight) or math.isinf(weight) or weight <= 0:
            return None
        return weight
    except (json.JSONDecodeError, ValueError, TypeError, OverflowError):
        return None

def main():
    print("==== Running calculate_dimensional_weight ====")
    dims_input = input("Enter dimensions LxWxH in cm (e.g. 20x30x10): ")
    dim_weight = calculate_dimensional_weight(dims_input)
    print("Dimensional Weight (kg):", dim_weight)

    print("\n==== Running format_tracking_label ====")
    code_input = input("Enter raw code (e.g. E123): ")
    label = format_tracking_label(code_input)
    print("Formatted Label:", label)

    print("\n==== Running parse_package_manifest ====")
    json_input = input('Enter manifest JSON (e.g. {"weight": 12.5}): ')
    weight = parse_package_manifest(json_input)
    print("Manifest Weight:", weight)

if __name__ == "__main__":
    main()
```

File 2: Updated test_processor.py

```
from hypothesis import given, strategies as st
from processor import calculate_dimensional_weight, format_tracking_label, parse_package_manifest

@given(st.text() | st.none())
def test_calculate_dimensional_weight_no_crash(s):
    try:
        calculate_dimensional_weight(s)
    except Exception as e:
        assert False, f"calculate_dimensional_weight crashed with: {e}"

@given(st.text() | st.none())
def test_format_tracking_label_safe(s):
    try:
        format_tracking_label(s)
    except Exception as e:
        assert False, f"format_tracking_label crashed with: {e}"

@given(st.text() | st.none())
def test_parse_package_manifest_safe(s):
    try:
        parse_package_manifest(s)
    except Exception as e:
        assert False, f"parse_package_manifest crashed with: {e}"
```